

Filtering and Sorting

Prof. Dr. Peter Braun

6. Dezember 2024

1 Titlepage

Filtering and Sorting

Prof. Dr. Peter Braun



No Text – maybe some music :-)

2 Learning Goals

Learning Goals

- You learn how to filter resources based on values.
- You learn how to combine filter criteria.
- You learn how to order the results.
- You learn how to apply hypermedia principles.

Welcome to this learning unit, in which we will look at filtering resources based on the values of attributes. After all, we can use two main approaches to reduce the number of resources the server provides in response to a request. In the first technique, we allow the client to load only a subset of the result. We then call these subsets pages, or pages in English, and the procedures for doing so pagination. We will talk about these techniques in another unit. Combined with this is the second technique we will deal with today. We will allow the client to select resources based on the values of individual attributes. The client can offer this to the end-user through the user interface, for example, in a search dialog where you can search for a person's first name, last name, or date of birth. We also need to discuss how the client can trigger complex search queries with logical links similar to a database query. Related to this search topic is the order in which we offer the results to the client and how the client can perhaps even change this order. In the end, we will come back to the topic of hypermedia. You already know this topic from many other units, and you will see at the end of this unit how the server informs the client via links that a search is possible at all.

3 Filter by One Attribute

Filter by One Attribute

```
1 @Path( "/persons" )
2 public class PersonService {
3     @GET
4     @Produces( MediaType.APPLICATION_JSON )
5     public Response getPersons(
6         @QueryParam( "firstname" ) String firstName )
7     {
8         List<Person> users = this.personDAO.loadByFirstName( firstName );
9         return Response.ok( users ).build();
10    }
11 }
```

■ `/api/persons?firstname=Eve`

■ `/api/persons` → parameter `firstName` is null

Filtering resources based on values already accompanies us through almost all units about Web APIs, so the source code on the slide should be easy to understand. You can see here an implementation for GET querying all people. In line six, we annotate the “first-name” parameter so that REST Easy knows to populate this parameter with the value from a query parameter from the URI. We pass the query for all people with a given first name to the database layer in line eight and return the result to the client in line nine. The client adds to the URI after the question mark, the name of the query parameter, the equal character, and the desired value. You already know this. However, if the client does not include a parameter, as in the last line on the slide, then the parameter “first-name” has the value NULL.

4 Working with Default Values

Working with Default Values

```
1 @Path( "/persons" )
2 public class PersonService {
3     @GET
4     @Produces( MediaType.APPLICATION_JSON )
5     public Response getPersons(
6         @DefaultValue( "" ) @QueryParam( "firstname" ) String firstName )
7     {
8         List<Person> users = this.personDAO.loadByFirstName( firstName );
9         return Response.ok( users ).build();
10    }
11 }
```

This special case could cause problems in the database layer. Therefore, using the additional annotation in line six, named “DefaultValue,” is good practice to define that the method parameter “first-name” should contain the empty string if the client does not set the query parameter itself. We have always used this in our examples in the last units.

5 Using Prefix and Full-text Search

Using Prefix and Full-text Search

- The server returns results even on single characters
- The attributes start with the given characters
- The attributes contain the characters

In the previous implementation, the server expects a person's full name from the client as a value in the query parameter. This name must be spelled correctly in terms of upper and lower case. Next, the server should generally be as tolerant as possible during a search. This means that incomplete values or spelling errors in the search should still lead to a result. We should first allow a first name to be written in lowercase letters only, and the server will still return the correct record. How could we further support the client or the end-user as a server? For a moment, please remember how you work with a search engine on the Web. When you type the first character, the search engine suggests possible completions to your query. So, referring to our example, if you search with the value J, the server should already return the record for the resource with the first name James. This behavior is called a prefix search. Instead of searching for the value of the query parameter only at the beginning of a specific attribute, the server could also offer a full-text search and return a record as soon as the entered character is present in the name. The server developer is free to make this decision; he does not have to document the implemented procedure to the client, either. However, he should, of course, consider which type of search makes sense for the use cases he is planning.

6 Query Parameters in JAX-RS

Query Parameters in JAX-RS

```
1 @Path( "/persons" )
2 public class PersonService {
3     @GET
4     @Produces( MediaType.APPLICATION_JSON )
5     public Response getPersons(
6         @DefaultValue( "" ) @QueryParam( "firstname" ) String firstName,
7         @DefaultValue( "0" ) @QueryParam( "age" ) int age,
8         @DefaultValue( "false" ) @QueryParam( "active" ) boolean active,
9         @DefaultValue( "" ) @QueryParam( "location" ) List<String> locs )
10    {
11        List<Person> users = this.personDAO.loadBy(...);
12        return Response.ok( users ).build();
13    }
14 }
```

- Types with a constructor that accepts a String value
- Types with static methods `fromString` or `valueOf`

In most units dealing with query parameters, the method parameter data type was String. Using the example beside me, which is only for demonstration purposes, I would like to show you that besides the String data type, we can use all primitive data types of the Java language, i.e., numbers, the two logical values for true and false, and even lists of such data types. Two more rules are described below the source code. In addition to the data types mentioned above, all data types, i.e., classes in Java, can be used to offer a constructor with only one parameter of type string. REST Easy reads the query parameter value from the Uri as a string and can then call the corresponding constructor of the data type. In addition, if such a constructor does not exist, REST Easy still looks for static methods named “fromString” and “valueOf” in the specified data types and would use these methods to translate a string into an object of that type. These rules allow us to use many standard Java classes as parameter types. Also, when implementing your data types, you can implement one possible way to use your class as a data type on a query parameter. In an emergency, for example, if you want to use a class for which all the ways just discussed should not be possible, there is still a way out via a converter. We will look at this way later.

7 Using LocalDate as Query Parameter Type

Using LocalDate as Query Parameter Type

```
1 @Path( "/persons" )
2 public class PersonService {
3     @GET
4     @Produces( MediaType.APPLICATION_JSON )
5     public Response getPersons(
6         @DefaultValue( "" ) @QueryParam( "bornafter" ) LocalDate bornAfter )
7     {
8         List<Person> users = personDAO.loadByBirthDateAfter(bornAfter);
9         return Response.ok( users ).build();
10    }
11 }
```

■ Example for a date as String: 2020-07-20

Let's return to the case when we want to use a data type for a query parameter, where all the rules presented so far for converting a string into an object of this data type should not apply. As an example, let's consider the "LocalDate" class. It provides a filter that allows us to pick out all people born after the specified date. When you enter the source code from the slide in your development environment, you will probably already see a warning saying that the datatype "LocalDate" is not allowed as a parameter. We are also dealing with a non-trivial problem here. First, REST Easy needs to learn to translate a string into an object of the datatype "LocalDate." However, more serious is that there are many different methods and some standards for formatting dates. REST Easy needs our support regarding which of these different date formatting should be allowed. By the way, this is a problem with the interface between the client and the server. Suppose we had settled on the date notation as seen on the slide below the source code, but how does the client know this exact date notation is correct? This would be the case for the documentation again, which is what we wanted to avoid. Another approach would be to assume that the client's developer already realizes that when formatting this date notation, he should use the same procedure that we use when serializing resources. When we started discussing the search, we mentioned that the server should be as tolerant of the values as possible. Related to our problem with the different date formatting, we could accept many other formats as a third option on the server. This would require more effort in implementing the server and would also cost some performance, but we could make a lovely offer to the clients. I also do this in my implementations and check about ten different date formats.

8 Implementing Parameter Converters

Implementing Parameter Converters

```
1 public class LocalDateConverter implements ParamConverter<LocalDate> {  
2     @Override public LocalDate fromString( final String value ) {  
3         return null; // TODO  
4     }  
5  
6     @Override public String toString( final LocalDate value ) {  
7         return null; // TODO  
8     }  
9 }
```

- Class `LocalDateConverterProvider` informs RestEasy about the existence of this converter
- The Provider must be annotated with `@Provider`

Here is an example of such a converter, which you must implement to convert a string into a Java object. With this converter, only some standard conversion rules work. JAX-RS or REST Easy provides an interface named “ParamConverter” and the two methods “fromString” and “toString.” You probably already know the principle if you have already dealt with serialization and deserialization. To inform REST Easy about this converter class, you need a class annotated as “Provider.”

9 Filter by Many Attributes: Logical And

Filter by Many Attributes: Logical And

```
1 @Path( "/persons" )
2 public class PersonService {
3     @GET
4     @Produces( MediaType.APPLICATION_JSON )
5     public Response getPersons(
6         @DefaultValue( "" ) @QueryParam( "firstname" ) String firstName,
7         @DefaultValue( "" ) @QueryParam( "lastname" ) String lastName )
8     {
9         List<Person> users =
10             this.personDAO.loadByNames( firstName, lastName );
11         return Response.ok( users ).build();
12     }
13 }
```

- `/api/persons?firstname=James&lastname=Bond`
- Most APIs understand this as logical And.
- The semantic is not clearly defined.

Now, let's move on to somewhat more complex searches. In the example, we are searching for the first and last names. Below the source code, you can see part of the URL the client must use to start such a search. The equal sign separates the query parameter and value as usual, and an ampersand separates the two query parameters. Because the ampersand is understood as logical and in programming languages, this search is usually understood as searching for a resource with the first name James and the last name Bond. The server returns a result only by combining these two values precisely. This understanding of the ampersand as logical is nowhere defined; it became only in such a way natural. As a server developer, you are not violating any standard if you interpret this URI differently, for example, as logical. The general question is whether it makes sense to specify multiple query parameters. Again, let's imagine how today's user interfaces are implemented in mobile or web applications. This possibility of two query parameters should be implemented in the user interface in a search dialog where the end-user can enter the desired values in two input fields. Please think about the last time you worked with such a search dialog. More common, because it is also much easier for the end-user to use, is a search bar, i.e., a single input field where the user enters the search criterion that is obvious to him, i.e., either a first name or a last name. Then, the backend performs the search using several attributes.

10 Supporting Search Bars

Supporting Search Bars

```
1 @Path( "/persons" )
2 public class PersonService {
3     @GET
4     @Produces( MediaType.APPLICATION_JSON )
5     public Response getPersons(
6         @DefaultValue( "" ) @QueryParam( "search" ) String search )
7     {
8         List<Person> users = this.personDAO.loadByNames( search );
9         return Response.ok( users ).build();
10    }
11 }
```

- It remains unclear for clients which attributes are used to search here.

- `/api/persons?search=jam`

The slide shows a possible implementation. Again, only one query parameter is offered, but we name it quite general, for example, Search. In line eight, we delegate the search to the database layer, which then searches for the desired value in many attributes. Let's assume the search was implemented using the resources' first and last names. Then, entering the query parameter Search with the value James would return all resources where the first or last name contains the value "James." You can combine this beautifully with the prefix search discussed earlier in this unit. Then, the URL you see on the slide at the bottom would return all resources where the string j a m occurs in either the first or last name. Another advantage of this general approach is that the search can be easily extended later in the backend without changing the interface to the client. After an update of the server, the client could then, for example, search using date specifications.

11 Filter By Many Values: Logical Or

Filter By Many Values: Logical Or

```
1 @Path( "/persons" )
2 public class PersonService {
3     @GET
4     @Produces( MediaType.APPLICATION_JSON )
5     public Response getPersons(
6         @DefaultValue( "" ) @QueryParam( "firstname" ) List<String> names )
7     {
8         List<Person> users = this.personDAO.loadByFirstNames( names );
9         return Response.ok( users ).build();
10    }
11 }
```

■ `/api/persons?firstname=Eve&firstname=James`

In the overview of the possibilities of which data types can be used as parameters in search queries, the collection of primitive data types or strings also occurred. I would now like to show you this again with an example. In line six, the query parameter “first-name” is defined. The method parameter has the data-type list of strings. You can now achieve that the query parameter may occur several times in the URI, as you see in the example below on the slide. Therefore, it makes sense to name the query parameter singular, even though many values will arrive in the method. For example, you could use this notation if the client wants to search for all resources, with the first name being Eve or James.

12 Comparison Operators

Comparison Operators

- Query parameters have strict syntax: `key=value`
- Not allowed: `key>value`
- `/api/persons?salaryGreaterThan=3000`
- `/api/persons?salary[gt]=3000`
- `/api/persons?q={"salary": {"$gt": 3000}}`

Now that you have seen how to create more complex queries using AND and OR logical operators, we also need to talk about how to use comparison operators. For example, let's assume that our service stores people's monthly salaries, and the server wants clients to query for people whose salary is greater than a given value. The syntax for sending query parameters and the values is clearly defined. An equal sign must separate query parameters and values. Notation, as you can see in the second point on the slide, is not allowed. However, the problem is not serious because we could rename the query parameter so that the type of comparison, greater than, becomes part of the name. This would allow us to write something you see in the list on the slide under item three. The interpretation of the query parameter is up to the server, and the server translates such a query into a corresponding database query with SQL. Suppose you don't want to offer only one kind of comparison operator but want to build the server so generically that you can make comparison operators possible for practically all query parameters. In that case, an approach like the one you can see in the second to last line on the slide is a good idea. After the query parameter salary, there is a textual representation of the comparison operator in square brackets. The abbreviation GT stands for greater than. Accordingly, you could also offer other relational operators, such as less than equal or unequal. The last approach on the slide is identical in content but a bit more flexible in notation. Here, the type of query is described in JSON format. You can imagine building very complex queries with comparison operators and logical operators of this type. By the way, this example comes from the REST interface of the MongoDB database.

13 Complex Query Languages

Complex Query Languages

- Should client developers learn query languages?
- `/persons?q=firstName=James+or+firstName=John`
- `/persons?q=firstName=James;age>=18`
- Defining your own query language is difficult.

However, the last example leads directly to the next question: Should we, as server developers, allow such complicated search queries at all? Two other examples of such complex search queries are beside me. In both examples, we use a generic query parameter called `q` to formulate a complex search query. However, the syntax is slightly different than in the previous examples. In the first example, we use the keyword `OR` to describe a search using the “first-name” attribute. Thus, the desired resources should contain either the value `James` or `John` in this attribute. The second example links two search queries by a semicolon, which is meant as a logical `AND`. We are searching for all people whose first name is `James` and who are of age. As a server developer, you can put much effort into defining your syntax for such complex search queries. Many freely available libraries define such languages or grammars. But my point is this: Should we require client developers to learn our language for search queries? More importantly, why does a client need such complicated search queries in the first place? As mentioned, this depends heavily on the use cases for which we have developed our server. However, the more complicated the query languages, the more we deviate from one of the basic principles of the HTTP paradigm. The server should make offers to the client on each request via hyperlinks in the response message about which requests to execute next. The client should be able to use these hyperlinks unchanged immediately. If we provide a complex language for search queries to the client, we can only tell the client that there is such a generic search parameter as `q`, for example, but we cannot give it an example of such a search query. Often, the provision of such complex search languages is also rather an excuse for the developer from the server because he wants to avoid dealing sufficiently with the possible use cases of an application. For example, the assertion is that one needs to learn how the client’s surface will be arranged and which kinds of search inquiries could be made in the

future. However, I'm afraid that's not right. These are precisely the considerations that a server developer must make when designing the interface. In a rest-based system, the server tells the client developer exactly what can be done with the interface.

14 Replace Query Parameters With Paths

Replace Query Parameters with Paths

- `/questions?tag=java` → `/questions/tagged/java`
- `/questions?status=open` → `/questions/open`
- `/questions?since=2020-10-01` → `/questions/latest`

Search queries should be simple for the client. Even though the server already gives the client information about what types of search queries can be made via the hyperlinks, the client still has to get the end user's specific input into the Uri. We'll look at how to do that in a moment. But before that, I want to show you that you can avoid using query parameters by modeling search queries using new path elements in many cases. You can see three examples on the slide. On the left, you see the Uri that uses query parameters; on the right, you see a suggestion of how the Uri could look with path elements. In terms of meaning, both are identical. In the first example, we are looking for queries that have been tagged with the keyword Java. Instead of the query parameter "tag," as shown on the left, we append another path element with the name tagged to the path element questions. Then, we add the desired keyword. Such Uris can be read hierarchically. The path element "questions" points to all resources of the type of question. Then, we only want to see those questions for which a keyword has been assigned, and this keyword is Java. The second example works the same way. Instead of the query parameter status, we offer a URI, where we first provide all resources of type question, then in the next step, all questions that are open in the status. The last example even shows how this approach can be used without concrete values in the query parameters, in this case, a date specification, and instead define a symbolic name as a new path element, in this case, the latest. This would make it even easier for the client to use this search because it can replace the concrete date specification, which it would have to add in the example on the left, with a constant URI. We would also have the freedom to change the definition for the newest questions on the server. The client would not have to be adapted at all.

15 Ordering

Ordering

- ❶ `/api/persons`
- ❷ `/api/persons?order=descending`
- ❸ `/api/persons/reverse`
- ❹ `/api/persons?order=+birthdate`
- ❺ `/api/persons?order=-birthdate`

Now, let's get to the issue of sorting orders. The server always returns the results of a query in some order. If the server developer should not have given any further thought, this is probably the order in which the data is in the database, i.e., sorted by ID. However, the server developer/should/ have thought about what order makes sense to the client. Often, a natural ordering of resources occurs based on attributes. People, for example, are sorted by last name and, as a second criterion, by first name. Resources with a time specification, and when that time specification is an important attribute, are sorted chronologically. Of course, you can also sort people by date of birth. It depends here, as always, on the use case. It is only important that the server developer thinks about this in advance and does not allow the client to sort the data arbitrarily via a general approach. With the examples beside me, I would like to offer you some possibilities. Let's assume that in the first example, the people would be returned sorted by last name and first name. The server developer has considered this sorting order reasonable in advance. He does not have to offer the client any other sorting order. The clients' developer should consider this and refrain from trying to reverse the order of the elements locally. To do this, he would have to load all resources first, or he would have to load the last page of the search result and then reverse the order in each case. The client must not implement if the server does not offer the function to reverse the sort order. However, let's assume the server developer wants to allow the client to load the resources in reverse order. Then, he would send the client a hyperlink in the response message to the first request, which would look similar to the second example on the slide. The client could then offer the end-user, for example, via a button, the action of making the request again, reversing the sort order. So, the query parameter to use here is "Order," and it would only accept the two values we use to distinguish between ascending and descending order. After what we just

heard about replacing query parameters with new path elements, you might now think a URI, like in the third example on the slide, would also be suitable. This is correct from a purely technical point of view, but this Uri is not particularly appropriate. But that is a matter of taste. What do we do now if the server wants to deliver the resources sorted by several criteria? For example, a table could display the results like a spreadsheet. And now, you want to allow the end-user to sort the results by the individual columns. Offering this is not forbidden in an HTTP-based system. The client mustn't have to assemble the URIs needed for each request. Provided that the server delivers the answer message hyperlinks, as in examples four and five, the client can also sort the results after the date of birth ascending and descending, and we did not hurt the hypermedia principle of REST. By the way, the plus or minus in front of the Birthday value is supposed to define the order, i.e., plus for ascending and minus for descending.

16 Hypermedia and Queries: URI Templates

Hypermedia and Queries: URI Templates

- Provide URI templates for queries
- `/api/persons?firstName={firstName}`
- This URI should be provided additionally.

Let's move on to the last topic for today. We just talked about the hypermedia principle in connection with the sort order and are now applying this principle to the search. Because we have yet to discuss one point, how does the client even know there are query parameters? Again, we must disclose that to the client's developer in some documentation. Fortunately, we can again use the hypermedia principle of HTTP. The server informs the client in the response message to a request in the form of a hyperlink about the possibility of using query parameters. However, this is not a complete and correct URI but a template. This is called URI templates in English; there is even a standard. You can see this example in the second line on the slide. The part of the address that the client should replace with a concrete value, for example, the input from an end-user, is enclosed in curly brackets. After all, we want to make it as easy as possible for the client to use URIs provided by the server. Therefore, in addition to this template, the server should always return the address to the client without any query parameters in case the client wants to load all elements and does not want to apply a search.

17 Providing URI Templates

Providing URI Templates

■ Use class `Hyperlinks` from the demo:

```
1 final Response.ResponseBuilder responseBuilder = Response.ok( );
2
3 Hyperlinks.addLink(
4     uriInfo,
5     responseBuilder,
6     "/persons?search={SEARCH}",
7     "searchForPersonByName",
8     "application/json"
9 );
10
11 responseBuilder.entity( allPersons );
12 return responseBuilder.build( );
```

A class called `Hyperlinks` in the repository provides a way to provide such hyperlinks in a response message. The slide shows an example of its use. In lines 3-8, we create a hyperlink consisting of the three ents: the relative part of the Uri, which is then automatically expanded to an absolute address, the relation name, and, as the last parameter, the media type. The first parameter in line four, is needed to determine the full address. The second parameter in line five is passed here so that the “addLink” method can immediately insert the new hyperlink into the response message.

18 Using URI Templates

Using URI Templates

- The client must replace all placeholders by the user-given input or the empty `String`:

```
1 uri = uri.replace( "{SEARCH}", "Jame" );
```

What would a client developer have to do now if he would like to translate such a template to the correct address, which he can use for the next request? Every programming language has a function or method to search and replace strings. Let's assume that the variable `URI` currently contains the template. The first parameter in the `replace` method call contains the placeholder; the second parameter contains the value with which this placeholder must be replaced. You will now realize that we have created a coupling or dependency between client and server. In this example, the placeholder's name, the string "Search," is stored as a constant in the client and may not be changed by the server. However, the concrete structure of the `Uri`, the names of the path elements, and the names of the query parameters may change. Thus, these placeholders now have the same meaning as the relation types in the hyperlinks, for which we also said that they might no longer change.

19 Summary

Summary

- You have learned how to use query parameters
- You have learned how to implement complex queries
- You have learned how to apply the Hypermedia principle

That's all for today. Today, you learned about another module we can use to make the interface between client and server practical and straightforward. In addition to dividing the result set into individual pages, filtering is the second important technique for creating a substantial result set manageable for the client. We can describe quite complex search queries based on the very simple concept of individual query parameters and their values. However, this topic has fewer standards and more best practices than in other areas. I have argued that, especially in a Web-based system, it is up to the server to decide which search queries and types of sort order are offered. In such a system, the client should generally not be allowed to make utterly arbitrary search queries using a new language. According to the hypermedia principle, always remember to inform the clients which search queries and sort orders are offered.

20 Literature

Literature

- Bill Burke: RESTful Java with JAX-RS 2.0: Designing and Developing Distributed Web Services (Englisch). O'Reilly, 2013.